InsaCloud

Les outils, brique par brique

Chaque technologie : ce qu'elle est, pourquoi elle et pas une autre, ce qu'elle fait ici

Ce document ne raconte pas le projet : il prend les outils un par un. Quarante-deux technologies, rangées en sept couches qui s'empilent. On part d'un dossier vide et, à chaque brique posée, la plateforme sait faire une chose de plus.

Pour chacune, quatre questions, toujours les mêmes : **c'est quoi**, **pourquoi celle-là et pas une autre**, **son rôle exact ici**, et **ce que son arrivée a changé**.

Projet 3 — *Outils de déploiement de plateformes*
INSA Centre-Val de Loire, STI 4A FISE, 2026/2027 — Dr. Nadia Fettah

13 octobre 2026

Table des matières

Comment lire ce document **1**

1 Couche 0 — Le socle **3**

2 Couche 1 — L'application web **6**

3 Couche 2 — Les données **12**

4 Couche 3 — Les machines louées **17**

5 Couche 4 — Le pilotage du parc **24**

6 Couche 5 — L'infrastructure **31**

7 Couche 6 — La qualité **36**

8 Récapitulatif **44**

8.1 Les quarante-quatre briques, d'un coup d'œil 45

8.2 Qui réalise quoi dans le cahier des charges 46

8.3 Les sept ajouts au-delà du sujet 46

8.4 Ce qu'il faut retenir 47

Comment lire ce document

Le principe : un empilement

On ne présente pas les outils par ordre alphabétique ni par importance. On les pose dans l'ordre où il a fallu les poser, et chaque couche ne peut exister que parce que la précédente est là.



Figure 1 – Les sept couches. On ne peut pas conteneuriser avant d'avoir une application, ni automatiser le déploiement avant d'avoir quelque chose à déployer.

Chaque outil, quatre questions

Toutes les fiches ont la même forme. Vous pouvez lire le document en entier, ou n'en consulter qu'une fiche.

C'est quoi en une ou deux phrases, sans supposer qu'on connaît.

Pourquoi celui-là quelles étaient les autres possibilités, et ce qui a fait pencher. C'est la partie la plus utile : un outil se comprend par ce qu'on a écarté.

Son rôle ici précisément, dans ce projet, avec le fichier concerné.

Ce que ça a changé ce que la plateforme savait faire avant, et après.

> **À retenir**
>
> Si vous ne devez retenir qu'une chose par outil, retenez le **pourquoi**

> **celui-là**. Savoir qu'on utilise Gunicorn ne vaut rien ; savoir pourquoi le serveur de développement de Flask ne suffit pas, si.

Ce que la plateforme doit savoir faire

Pour mesurer l'apport de chaque brique, il faut savoir où l'on va. La plateforme finale doit :

1. accueillir des comptes utilisateurs et les authentifier ;
2. proposer un catalogue de systèmes Linux ;
3. créer une machine à la demande, en quinze secondes ;
4. en donner l'accès par SSH, par un terminal web et par un bureau graphique ;
5. la détruire toute seule à l'échéance ;
6. répartir les machines sur plusieurs serveurs ;
7. survivre à la panne de l'un d'eux ;
8. et se redéployer entièrement en une commande.

Aucune de ces huit capacités n'est apportée par un seul outil. Chacune naît de la rencontre de plusieurs briques — et c'est précisément ce que ce document cherche à rendre lisible.

CHAPITRE 1

Couche 0 — Le socle

Avant la première ligne de code métier, trois outils décident de ce que le projet pourra devenir. On les oublie vite parce qu'ils ne « font » rien de visible, et ce sont pourtant eux qu'on regrette quand ils manquent.

Git

brique 1 · versionnement

C'est quoi

Un système qui enregistre l'historique complet d'un dossier : qui a changé quoi, quand, et pourquoi. Chaque enregistrement — un *commit* — est une photographie du projet entier, pas seulement des fichiers modifiés.

Pourquoi celui-là

L'alternative honnête, c'est de copier le dossier en `projet-v2-final-vraiment`. Ça marche jusqu'au jour où l'on veut savoir quand un bug est apparu, ou revenir trois jours en arrière sans perdre le travail d'hier.

Il existe d'autres systèmes — Subversion, Mercurial — mais Git est celui que toute l'industrie utilise, et surtout le seul que les plateformes d'intégration continue savent lire nativement. Choisir autre chose aurait coupé le projet de GitHub Actions, donc de toute la couche 6.

Son rôle ici

Trente-cinq commits, chacun avec un message qui explique *pourquoi* et pas seulement *quoi*. Les branches servent à isoler un travail en cours : la branche `aziz-youssef-modif` porte l'intégration d'une contribution extérieure sans toucher à `main` tant qu'elle n'est pas vérifiée.

Un fichier `.gitignore` tient hors du dépôt ce qui ne doit pas y entrer : les clés (`.secret_key`, `.vault_key`), le fichier `.env` des secrets de la pile, les environnements virtuels, et les archives de travail des utilisateurs.

Ce que ça a changé

Sans Git, aucune des briques de la couche 6 n'existe : pas d'intégration continue, pas de recherche de secrets dans l'historique, pas de déploiement déclenché par un envoi de code. C'est la brique dont toutes les autres dépendent indirectement.

Python 3.10

brique 2 · langage

C'est quoi

Le langage dans lequel est écrite toute la logique du projet : l'application web, le démon de nettoyage, l'agent des nœuds, les scripts de génération de schémas.

Pourquoi celui-là

Trois contraintes ont guidé le choix.

D'abord la **bibliothèque standard**. L'agent qui tourne sur chaque worker ne doit rien avoir à installer : il utilise `urllib` pour les appels HTTP, `json`, `ssl`, `subprocess`, et lit `/proc/meminfo` directement. En Node.js ou en Go, il aurait fallu déployer un binaire ou un environnement sur chaque nœud.

Ensuite l'**écosystème** : Flask, SQLAlchemy, Ansible lui-même sont en Python. Rester dans un seul langage évite d'avoir deux chaînes d'outillage.

Enfin la **version** : 3.10 précisément, parce que la cible de déploiement est Ubuntu 22.04, qui fournit cette version. Ce détail nous a coûté une panne : nous avions épinglé une bibliothèque qui n'existait pas pour 3.10, les tests passaient sur nos postes et le déploiement échouait.

> **Piège — La version du poste n'est pas celle de la cible**
>
> C'est l'erreur la plus banale et la plus coûteuse. La chaîne d'intégration force désormais `PYTHON_VERSION: 3.10` : elle teste avec le Python de la machine de production, pas celui du développeur.

Son rôle ici

Environ 3 500 lignes pour l'application, 1 600 pour les tests, plus les scripts d'outillage. Le code est séparé en modules dont chacun ignore les couches au-dessus de lui : `docker_ops.py` ne sait pas que Flask existe, ce qui le rend testable sans serveur.

Ce que ça a changé

Un seul langage du navigateur au worker, et un agent déployable sans rien installer.

venv et requirements épinglés brique 3 · reproductibilité

C'est quoi

Un *environnement virtuel* est un dossier contenant une copie isolée de Python et de ses bibliothèques. Le fichier `requirements.txt` liste les bibliothèques avec leur **version exacte** : `Flask==3.1.3`, pas `Flask`.

Pourquoi celui-là

Sans isolation, installer une bibliothèque pour ce projet peut casser un autre projet sur la même machine. Sans versions figées, le projet marche

aujourd'hui et casse demain parce qu'une dépendance a publié une version incompatible.

L'alternative moderne serait Poetry ou uv, plus confortables. On est resté sur `venv` et `pip`, qui sont dans la bibliothèque standard : aucune installation préalable n'est nécessaire sur la machine cible, et le rôle Ansible peut créer l'environnement avec ce que fournit Ubuntu.

Son rôle ici

Deux fichiers : `requirements.txt` pour ce qui tourne en production, `requirements-dev.txt` pour l'outillage de tests et d'analyse. Le rôle Ansible `webapp` installe exactement le premier sur le contrôleur — pas une liste recopiée à la main, ce qui a déjà été une source de divergence.

Ce que ça a changé

C'est ce qui permet à `pip-audit` d'exister (couche 6) : on ne peut confronter des dépendances à une base de vulnérabilités que si l'on sait exactement lesquelles on utilise.

Ce que la plateforme sait faire après la couche 0

Rien d'utilisable. Mais le projet est désormais *reproductible* : on peut le cloner, créer l'environnement, et obtenir exactement les mêmes versions. Sans cela, toutes les couches suivantes reposeraient sur du sable.

CHAPITRE 2

Couche 1 — L'application web

Six briques pour transformer du code Python en un site que des gens peuvent utiliser. Chacune règle un problème que la précédente laissait ouvert.



Figure 2.1 – Le chemin d'une requête. Chaque maillon existe parce que le suivant ne sait pas faire son travail.

Flask

brique 4 · le cadre web

C'est quoi

Un *micro*-framework : il fournit le routage (associer une URL à une fonction), les gabarits et les sessions. Il ne fournit ni base de données, ni administration, ni authentification — on choisit soi-même.

Pourquoi celui-là

L'autre grand choix Python est **Django**, qui fournit tout : ORM, interface d'administration, comptes utilisateurs, formulaires. Pour un projet d'entreprise classique, Django va plus vite.

Ici, c'était un mauvais choix pour une raison précise : le sujet demande de comprendre et de montrer chaque mécanisme. Avec Django, l'authentification est une boîte noire qu'on active ; avec Flask, on écrit le hachage, la session, le verrouillage après cinq échecs, et on peut les expliquer. Le projet est un exercice d'ingénierie, pas une course à la livraison.

Deuxième raison : Django impose son ORM. Nous voulions pouvoir piloter la base finement — notamment l'index unique partiel qui garantit l'unicité des ports.

Son rôle ici

`app.py` porte les routes HTML et les sessions ; `api.py` porte les routes JSON. Une particularité : **les mêmes URL servent les deux**, selon l'entête `Accept` de l'appelant. Il n'y a pas un site d'un côté et une API de l'autre, mais une logique métier unique avec deux rendus.

Listing 2.1 — 2_webapp/app.py

```
def veut_json() -> bool:
    # Un client qui accepte TOUT (Accept: */*, cas de curl sans en-tete
        ↪ )
    # n'est pas un client d'API : seul celui qui demande du JSON sans
        ↪ accepter
    # le HTML est traite comme tel.
    accepte = request.accept_mimetypes
    return accepte.accept_json and not accepte.accept_html
```

Ce que ça a changé

La plateforme devient atteignable depuis un navigateur. Et, grâce à la négociation de contenu, elle est simultanément pilotable par `curl` sans une ligne de code en plus — ce qui satisfait l'exigence d'API du sujet sans dédoubler le travail.

Jinja2

brique 5 · les gabarits

C'est quoi

Le moteur qui remplit des fichiers HTML avec des variables. Dans le gabarit on écrit `{{ machine.nom }}` et `{% for m in machines %}` ; Jinja2 produit le HTML final.

Pourquoi celui-là

L'alternative serait de construire le HTML par concaténation de chaînes en Python. Outre l'illisibilité, c'est une faille : une variable contenant `<script>` serait injectée telle quelle dans la page. Jinja2 **échappe automatiquement** tout ce qu'il insère — c'est une protection contre les injections de script obtenue sans y penser.

Il vient avec Flask, donc aucun choix à faire ; mais le mentionner compte, parce que l'échappement automatique est une des raisons pour lesquelles le projet n'a aucune faille XSS.

Son rôle ici

Les gabarits de 2_webapp/templates/ : la page de connexion, l'inscription, et le tableau de bord qui fait à lui seul 358 lignes. Les conditions y servent à n'afficher une section que si elle a du contenu — « Archives de mon travail » n'apparaît que s'il y a des archives.

Ce que ça a changé

Des pages lisibles à écrire, et une classe entière d'attaques écartée par défaut.

Werkzeug

brique 6 · la plomberie HTTP

C'est quoi

La bibliothèque sur laquelle Flask est construit. Elle gère le détail du protocole HTTP, et fournit deux outils qu'on utilise directement : le hachage des mots de passe et le *middleware* `ProxyFix`.

Pourquoi celui-là

On ne l'a pas choisie : elle arrive avec Flask. Mais deux de ses fonctions sont employées délibérément plutôt que réécrites.

`generate_password_hash` applique **pbkdf2-sha256** avec un sel aléatoire et des milliers d'itérations. Écrire soi-même un hachage de mot de passe est l'erreur classique : on choisit un algorithme trop rapide, on oublie le sel, et la base devient exploitable en cas de fuite.

`ProxyFix` règle un problème qu'on ne voit qu'en production : derrière nginx, toutes les requêtes semblent venir de `127.0.0.1`. Sans lui, notre verrouillage anti-force-brute compterait toutes les tentatives comme venant de la même adresse — et bloquerait tout le monde au cinquième échec de n'importe qui.

Son rôle ici

Hachage à l'inscription et vérification à la connexion ; `ProxyFix` activé dans `app.py` pour que l'adresse réelle du client remonte.

Ce que ça a changé

Des mots de passe stockés correctement, et un anti-force-brute qui vise la bonne personne.

Gunicorn

brique 7 · le serveur d'application

C'est quoi

Un serveur WSGI : il reçoit les requêtes HTTP et les passe à l'application Python. Il lance plusieurs processus ouvriers qui se partagent le travail.

Pourquoi celui-là

Flask embarque un serveur de développement, mais il traite **une requête à la fois** et affiche lui-même un avertissement : ne pas utiliser en production. Or créer un conteneur prend plusieurs secondes ; avec un seul processus, un utilisateur qui loue une machine bloquerait tous les autres.

Les alternatives sont uWSGI (plus puissant, bien plus complexe à configurer) et Waitress (plus simple, mais mono-processus par défaut). Gunicorn est le milieu raisonnable : une ligne de commande suffit, et il redémarre les ouvriers qui plantent.

Son rôle ici

Quatre ouvriers, lancés par l'unité systemd du contrôleur :

```
gunicorn --preload -w 4 -b 127.0.0.1:5000 app:app
```

Piège — Quatre processus qui démarrent en même temps

Chacun importe l'application, qui crée les tables si elles manquent. Les quatre le font simultanément, et parfois l'un échoue avec `table users already exists`. Sur un poste mono-processus, le bug n'apparaît jamais.

Deux correctifs : la création tolère désormais l'erreur « existe déjà », et l'option `--preload` fait importer l'application *une fois* avant de dupliquer le processus. Nous avons reproduit la panne avec huit processus simultanés — six sur huit démarraient ; après correction, huit sur huit.

Ce que ça a changé

La plateforme supporte plusieurs utilisateurs simultanés. Avant Gunicorn, une location en cours gelait le site pour tout le monde.

nginx

brique 8 · le proxy inverse

C'est quoi

Un serveur web placé *devant* l'application. Le navigateur lui parle ; il relaie vers Gunicorn.

Pourquoi celui-là

Gunicorn sait répondre à du HTTP, mais ne sait pas faire trois choses essentielles : terminer le TLS, servir efficacement les fichiers statiques, et absorber les requêtes malformées.

L'alternative est Apache, qui sait tout faire aussi mais consomme davantage et se configure plus lourdement. Caddy aurait apporté les certificats automatiques, ce qui n'a pas de sens sur un réseau privé sans nom de domaine public.

Son rôle ici

Le rôle Ansible `reverse_proxy` installe nginx avec :

- **TLS 1.3** et le certificat du nœud ;
- les fichiers statiques servis directement, sans réveiller Python ;
- les en-têtes de sécurité : `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options` ;
- `X-Forwarded-For`, que `ProxyFix` relira côté Flask.

Ce que ça a changé

Le site passe en HTTPS, les images et le CSS ne coûtent plus de temps Python, et l'application n'est plus exposée directement au réseau.

Tailwind CSS, précompilé

brique 9 · l'apparence

C'est quoi

Une bibliothèque de style où l'on compose l'apparence avec de petites classes dans le HTML, plutôt qu'en écrivant du CSS à part.

Pourquoi celui-là

L'usage courant est de charger Tailwind depuis un CDN : une ligne, et ça marche. C'était impossible ici.

La politique de sécurité de contenu du projet est `default-src 'self'` : **aucune ressource externe** n'a le droit d'être chargée. Un CDN est précisément une ressource externe. Il a donc fallu **précompiler** Tailwind en un fichier CSS servi par nginx, et héberger les polices localement.

Son rôle ici

`2_webapp/static/css/tailwind.css` est le résultat de la compilation ; `tailwind.src.css` en est la source. Les polices IBM Plex Sans sont dans `static/fonts/`.

Ce que ça a changé

Une interface soignée *sans* affaiblir la politique de sécurité. C'est le genre de compromis où le choix facile (le CDN) aurait coûté une faille entière : avec un CDN autorisé, une injection de script redevient exploitable.



Figure 2.2 – La page de connexion. Tout le style vient d'un fichier CSS local : la politique de sécurité interdit toute ressource externe.

Ce que la plateforme sait faire après la couche 1

Elle a un site en HTTPS, capable de servir plusieurs utilisateurs, avec des comptes et des sessions. Mais elle ne retient rien : à chaque redémarrage, tout disparaît. C'est l'objet de la couche suivante.

CHAPITRE 3

Couche 2 — Les données

L'application a des comptes et des sessions, mais elle oublie tout au redémarrage. Cinq briques pour qu'elle se souvienne — et pour que ce dont elle se souvient reste secret.

SQLAlchemy 2.0

brique 10 · l'ORM

C'est quoi

Un *Object-Relational Mapper* : il fait correspondre une classe Python à une table SQL. On écrit des objets, il écrit le SQL.

Pourquoi celui-là

L'alternative est d'écrire le SQL à la main. Trois raisons l'ont écartée.

Les injections SQL. Écrire `f"SELECT * FROM users WHERE nom='{saisie}'"` laisse un utilisateur nommé `' OR '1'='1` lire toute la table. L'ORM passe toujours par des paramètres liés : l'injection devient structurellement impossible, pas seulement évitée par discipline.

Deux moteurs. Le projet tourne sur SQLite en développement et sur PostgreSQL en production. Leur SQL diffère sur les dates, les types, les clauses. L'ORM écrit le dialecte qui convient.

La vérification. Une faute de frappe dans un nom de colonne devient une erreur Python immédiate, au lieu d'une erreur SQL découverte en production.

On aurait pu prendre l'ORM de Django, mais il impose Django. Celui de SQLAlchemy est le standard du monde Python hors Django, et sa version 2.0 apporte une écriture typée que les analyseurs comprennent.

Son rôle ici

`2_webapp/models.py` décrit les sept tables. Chaque colonne est annotée pour que `ruff` et l'éditeur sachent son type :

```
class Rental(Base):
    __tablename__ = "rentals"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("users.id"),
        ↪ nullable=False)
    end_time: Mapped[datetime] = mapped_column(DateTime, nullable=False
        ↪ )
    __table_args__ = (Index("idx_rentals_end", "status", "end_time"),)
```

L'index de la dernière ligne n'est pas décoratif : c'est exactement la requête que le Faucheur exécute toutes les dix secondes.

Ce que ça a changé

La plateforme se souvient. Et une famille entière d'attaques — l'injection SQL — disparaît du périmètre.

Alembic

brique 11 · les migrations

C'est quoi

L'outil qui fait évoluer le schéma d'une base *sans perdre les données*. Chaque changement devient un fichier numéroté qui connaît son prédécesseur ; la base retient où elle en est.

Pourquoi celui-là

Tant qu'on développe, on efface la base et on recommence. Dès qu'il y a des données réelles — des comptes, des locations en cours — c'est impossible.

Sans Alembic, la seule option serait d'écrire les `ALTER TABLE` à la main et de se souvenir lesquels ont été appliqués où. C'est exactement le genre de mémoire humaine qui échoue au pire moment.

Son rôle ici

`2_webapp/migrations/` contient deux révisions : le schéma initial, puis l'ajout de la table `archives`. La chaîne d'intégration exécute `alembic check`, qui échoue si les modèles et les migrations divergent — impossible d'oublier de générer une migration.

Piège — SQLite ne sait pas modifier une colonne

PostgreSQL accepte `ALTER TABLE ... ALTER COLUMN`. SQLite, non : il faut créer une table neuve, recopier, supprimer, renommer. Alembic sait le faire, mais seulement si on le lui demande par `render_as_batch=True`. Sans cette option, les migrations passent sur un moteur et échouent sur l'autre.

Ce que ça a changé

Le schéma peut évoluer en production. C'est ce qui a permis d'ajouter la sauvegarde du travail — une table de plus — sur une base contenant déjà des comptes et un historique.

SQLite

brique 12 · la base de développement

C'est quoi

Une base de données contenue dans un seul fichier, sans serveur à lancer.

Pourquoi celui-là

Pour développer et pour les tests, démarrer un serveur PostgreSQL est une friction : il faut l'installer, le configurer, le lancer. SQLite est déjà dans Python.

Le projet l'utilise en mode **WAL** (*write-ahead logging*), qui permet à plusieurs processus de lire pendant qu'un autre écrit — nécessaire puisque l'application et le Faucheur touchent la même base.

Son rôle ici

Base par défaut quand `DATABASE_URL` n'est pas défini. Les 168 tests l'utilisent : chacun part d'un fichier vierge dans un dossier temporaire, ce qui garantit qu'aucun test n'hérite de l'état d'un autre.

Ce que ça a changé

Les tests tournent en vingt secondes sur n'importe quelle machine, sans service à installer. C'est ce qui rend l'intégration continue légère.

PostgreSQL 16

brique 13 · la base de production

C'est quoi

Un serveur de base de données complet, conçu pour plusieurs clients simultanés.

Pourquoi celui-là

SQLite suffit à un processus, pas à quatre ouvriers Gunicorn plus le Faucheur qui écrivent en même temps. Son verrouillage porte sur le fichier entier : sous charge, les écritures s'attendent.

MySQL aurait fait l'affaire ; PostgreSQL a été préféré pour deux raisons concrètes ici : les **index partiels** — la clause `WHERE` sur un index unique, qui garantit l'unicité des ports sans bloquer leur réutilisation — et le respect strict des types.

Son rôle ici

Service `db` de la pile compose, image `postgres:16-alpine`. Son port **n'est pas publié** : seuls les services du réseau interne y accèdent. Il démarre avec `--data-checksums`, qui détecte une corruption silencieuse du disque.

Ce que ça a changé

La plateforme supporte la concurrence réelle. Et l'index unique partiel devient une garantie du moteur, pas une intention du code.

cryptography · Fernet

brique 14 · le coffre

C'est quoi

Fernet est une recette de chiffrement symétrique authentifié : elle combine AES en mode CBC et un HMAC. On ne peut ni lire sans la clé, ni modifier sans que cela se voie.

Pourquoi celui-là

Le problème est particulier : le mot de passe root d'une machine louée doit pouvoir être **réaffiché**. On ne peut donc pas le hacher — le hachage est à sens unique. Il faut un chiffrement réversible.

Écrire soi-même du chiffrement est l'erreur classique : on choisit mal le mode, on réutilise un vecteur d'initialisation, on oublie l'authentification. Fernet n'a **aucun paramètre à choisir** : c'est précisément son intérêt pour qui n'est pas cryptographe.

> **Notion — Hacher ou chiffrer : deux besoins, deux outils**
>
> Le mot de passe d'un **compte** est **haché** : on ne doit jamais pouvoir le retrouver, seulement vérifier qu'un essai donne la même empreinte.
> Le mot de passe **root d'une machine louée** est **chiffré** : il doit pouvoir être réaffiché à son propriétaire.
> Hacher le second le rendrait inutilisable ; chiffrer le premier serait une faute.

Son rôle ici

`2_webapp/crypto.py` charge deux clés **distinctes** : celle qui signe les sessions Flask, et celle qui chiffre les mots de passe root. Les séparer garantit qu'un vol de la première ne donne pas accès aux machines.

Les clés viennent de l'environnement — Ansible les injecte depuis son Vault — ou, à défaut, d'un fichier local en permissions `0600`.



Figure 3.1 – Le mot de passe root, affiché une seule fois. Après cet instant il n'existe plus qu'en base, chiffré.



Figure 3.2 – Le coffre rouvert après re-saisie du mot de passe de compte : les accès redeviennent visibles cinq minutes, puis se remasquent.

Piège — Une session Flask est signée, pas chiffrée

Notre première version déposait le mot de passe root dans le cookie de session, le temps de l'afficher. Or le contenu d'un cookie de session Flask est encodé en base64 : **n'importe qui peut le lire** sans la clé. La signature empêche seulement de le modifier.
On ne stocke plus que l'identifiant de la machine ; le secret est relu en base et déchiffré au moment de l'affichage, après vérification du propriétaire.

Ce que ça a changé

Un secret qu'on peut revoir sans jamais le stocker en clair, et qui ne transite plus par le navigateur.

Ce que la plateforme sait faire après la couche 2

Elle retient les comptes, les locations et les machines ; elle sait faire évoluer son schéma sans perdre de données ; et elle garde des secrets qu'elle peut réafficher. Il lui manque l'essentiel : les machines à louer.

CHAPITRE 4

Couche 3 — Les machines louées

C'est le cœur du produit. Sept briques pour fabriquer ce que l'utilisateur achète : une machine Linux à lui, joignable de trois façons.

Docker

brique 15 · la conteneurisation

C'est quoi

Un moteur qui exécute des processus isolés les uns des autres, en réutilisant le noyau de la machine hôte. Un *conteneur* se croit seul au monde, mais ne porte pas de système d'exploitation complet.

Pourquoi celui-là

La vraie alternative était la machine virtuelle — VirtualBox, KVM. Une VM offre une isolation plus forte, puisqu'elle a son propre noyau. Deux chiffres ont tranché :

	Machine virtuelle	**Conteneur**
Démarrage	30 à 60 secondes	moins d'une seconde
Mémoire minimale	plusieurs centaines de Mo	celle des processus
Densité sur un serveur	quelques dizaines	plusieurs centaines

Pour une location **à la minute**, faire attendre l'utilisateur une minute est absurde. Le sujet tranche d'ailleurs explicitement : « des conteneurs dans notre cas ».

Notion — Ce qu'un conteneur est réellement

Trois mécanismes du noyau Linux, pas un objet magique :
Les espaces de noms cloisonnent ce que le processus *voit* : son propre arbre de processus, ses propres interfaces réseau, sa propre vision du disque. Le port 22 du conteneur n'est pas le port 22 de l'hôte.
Les groupes de contrôle limitent ce qu'il *consomme* : mémoire, processeur, nombre de processus.
Les capacités découpent les pouvoirs de root en une quarantaine de droits indépendants, qu'on accorde un par un.

Son rôle ici

Six images, et une commande imposée par le sujet pour créer chaque machine :

```
docker run -d --restart=always -p <port>:22 <image>
```

Le projet y ajoute le durcissement : `--cap-drop ALL` puis les dix capacités strictement nécessaires, `no-new-privileges`, une limite de processus, de CPU, et la mémoire en réservation *et* en limite.

Ce que ça a changé

La plateforme sait livrer une machine en quinze secondes. C'est la brique qui rend le produit possible.

Dockerfile

brique 16 · la recette des images

C'est quoi

Un fichier texte qui décrit, instruction par instruction, comment construire une image. Chaque instruction crée une *couche* empilée sur la précédente.

Pourquoi celui-là

On pourrait créer une image en lançant un conteneur, en l'installant à la main, puis en figeant le résultat. Personne ne saurait plus ce qu'il contient, ni comment le reproduire.

Le Dockerfile est l'infrastructure comme code appliquée aux images : versionné, relu, et vérifiable par un analyseur.

Son rôle ici

Quatre fichiers pour six images. Le mode bureau n'est pas un fichier séparé : c'est un paramètre de construction.

```
FROM ubuntu:22.04
ARG DESKTOP=0
RUN apt-get update \
    && apt-get install -y --no-install-recommends openssh-server
        ↪ supervisor \
    && rm -rf /var/lib/apt/lists/*
COPY entrypoint.sh /usr/local/bin/entrypoint.sh
CMD ["/usr/local/bin/entrypoint.sh"]
```

Piège — Nettoyer dans une autre couche ne sert à rien

Si l'on installe des paquets dans un `RUN` et qu'on efface le cache d'apt dans le `RUN` suivant, le cache reste dans l'image : une couche ne peut pas alléger celle du dessous. Il faut enchaîner les deux dans la *même*

instruction. Sur nos images, cela représente une cinquantaine de mégaoctets.

Piège — Un mot de passe par défaut dans l'image

Notre première version contenait `ENV ROOT_PASSWORD=insacloud`. L'analyseur `hadolint` l'a signalé, et il avait raison : ce mot de passe aurait été le même sur **toutes** les machines louées, et visible par quiconque fait `docker history`. L'entrypoint en tire désormais un au hasard si la plateforme n'en fournit pas.

Ce que ça a changé

Six images reproductibles, construites par la chaîne d'intégration à chaque envoi, et analysées avant d'être publiées.

supervisord

brique 17 · plusieurs services dans un conteneur

C'est quoi

Un gestionnaire de processus : il en démarre plusieurs, les surveille, et relance celui qui meurt.

Pourquoi celui-là

C'est la brique qui résout une contradiction. Un conteneur exécute *un* processus, et s'arrête quand celui-ci se termine. Or une machine louée doit faire tourner cinq services : SSH, le terminal web, le serveur d'affichage, le bureau, et la passerelle VNC.

Les alternatives : lancer un script shell qui démarre tout en arrière-plan — mais alors rien ne surveille, et un service mort le reste. Ou `systemd` dans le conteneur — lourd, et il réclame des privilèges qu'on vient justement de retirer.

Notion — Pourquoi le PID 1 n'est pas un processus comme un autre

Le premier processus d'un conteneur hérite d'une obligation : adopter les processus orphelins et récupérer leur code de sortie. Sans cela, les processus terminés s'accumulent à l'état *zombie* et finissent par saturer la table.
Un `bash` lancé comme PID 1 ne fait pas ce travail. `supervisord`, si.

Son rôle ici

L'entrypoint écrit une configuration adaptée au mode choisi, puis se **remplace** par supervisord :

```
exec "$(command -v supervisord)" -c "$CONF"
```

Le mot `exec` compte : sans lui, le shell resterait PID 1 et ne relaierait pas les signaux. Un `docker stop` deviendrait un arrêt brutal au bout du délai de grâce.

Ce que ça a changé

C'est le premier des trois niveaux de résilience du projet : si un service meurt *dans* la machine, il est relancé sans que l'utilisateur s'en aperçoive.

OpenSSH

brique 18 · l'accès demandé par le sujet

C'est quoi

Le serveur SSH : il donne un terminal distant chiffré, authentifié par mot de passe ou par clé.

Pourquoi celui-là

Il n'y avait pas de choix à faire : c'est l'accès que le sujet impose, et c'est le standard universel. La question intéressante est plutôt *comment* on l'expose.

Le conteneur écoute sur son propre port 22, et Docker le publie sur un port de l'hôte dans la plage 8000–9000. C'est ce qui permet de faire tourner des dizaines de serveurs SSH sur une seule machine : chacun croit avoir le port 22.

Son rôle ici

Deux modes d'authentification. Par défaut, le mot de passe root généré. Mais si l'utilisateur a enregistré sa clé publique sur son compte, la machine **n'accepte plus que** cette clé — l'authentification par mot de passe est coupée.

> **Piège — Le port est publié avant que le service écoute**
>
> Docker publie le port dès la création du conteneur, mais `sshd` met une seconde ou deux à démarrer. Un test qui se connecte immédiatement échoue de façon intermittente.
> La correction n'est pas d'attendre « assez longtemps » — une durée fixe est toujours trop courte un jour — mais de réessayer jusqu'à ce que le service réponde, avec une limite.

Ce que ça a changé

L'accès exigé par le cahier des charges, avec une option plus sûre que le mot de passe pour qui le souhaite.

ttyd

brique 19 · le terminal dans le navigateur

C'est quoi

Un petit serveur qui expose un terminal dans une page web, par WebSocket.

Pourquoi celui-là

Le sujet ne le demande pas. Il répond à un problème réel : un utilisateur sur un poste de salle de TP n'a pas toujours de client SSH, ni le droit d'en installer.

Les alternatives — Shellinabox, abandonné ; Wetty, en Node.js donc une chaîne d'outillage de plus — ont été écartées. ttyd est un binaire unique, en C, de quelques centaines de kilooctets.

Son rôle ici

Lancé par supervisord sur le port 7681 du conteneur, publié sur l'hôte. Détail qui compte : il ne lance pas un shell, il lance `login`.

```
ttyd -p 7681 -W /bin/sh -c "cat /etc/issue; exec $(command -v login)"
```

Le port ne donne donc pas un accès ouvert : il présente une invite qui réclame l'identifiant et le mot de passe.

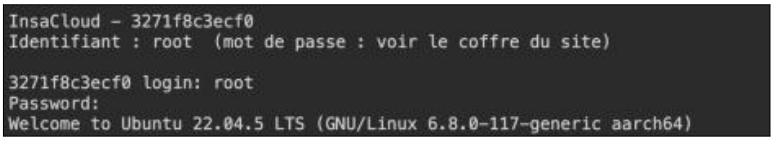


Figure 4.1 – Ce que montre le port du terminal web : une invite de connexion, pas un shell.

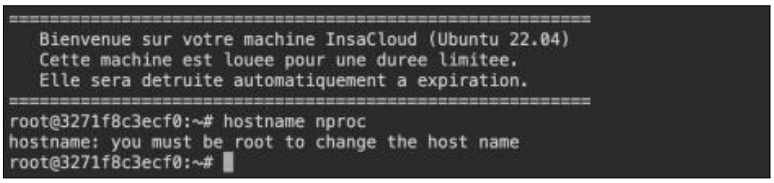


Figure 4.2 – Après authentification, le bandeau rappelle que la machine est louée pour une durée limitée.

Ce que ça a changé

Un accès sans rien installer. C'est le premier des ajouts au-delà du sujet.

Xvnc, XFCE et noVNC

brique 20 · le bureau graphique

C'est quoi

Trois pièces qui s'emboîtent. **Xvnc** est un serveur d'affichage qui dessine dans une mémoire plutôt que sur un écran. **XFCE** est l'environnement de bureau qui s'y affiche. **noVNC** est une passerelle qui traduit le protocole VNC en WebSocket, pour qu'un navigateur puisse l'afficher.

Pourquoi celui-là

XFCE plutôt que GNOME ou KDE : il tourne correctement dans un gigaoctet de mémoire, là où les autres en réclament deux ou trois. Pour une machine louée, c'est la différence entre trois utilisateurs par serveur et dix.

noVNC plutôt qu'un client VNC à installer : même raisonnement que pour ttyd — l'utilisateur ne doit rien installer.

Son rôle ici

Trois services de plus sous supervisord, en mode bureau seulement. Le serveur d'affichage **n'écoute que sur la boucle locale** : on ne peut l'atteindre qu'à travers la passerelle, qui est chiffrée quand un certificat est monté.

Le mot de passe VNC est limité à huit caractères par le protocole : ce sont les huit premiers du mot de passe root, ce que l'interface explique.

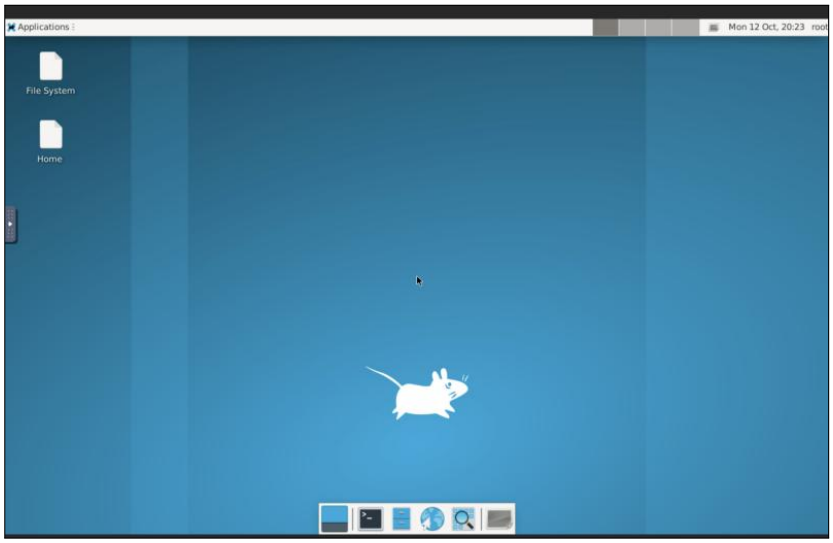


Figure 4.3 – Le bureau XFCE d'une machine Ubuntu louée, affiché dans le navigateur. Cinq processus cohabitent dans ce conteneur.

Ce que ça a changé

Le second ajout au-delà du sujet, et le plus spectaculaire en démonstration : un bureau Linux complet, dans un onglet, sans rien installer.

Le script d'entrée

brique 21 · ce qui assemble la machine

C'est quoi

`1_docker/entrypoint.sh`, 166 lignes de shell. C'est le programme que Docker lance quand la machine démarre.

Pourquoi celui-là

Il fallait un endroit unique qui adapte la machine à ce que l'utilisateur a demandé. Mettre cette logique dans le Dockerfile aurait obligé à six images indépendantes ; la mettre dans l'application aurait dispersé le savoir.

Son rôle ici

Trois tâches, dans l'ordre :

1. poser le mot de passe root et, s'il y en a une, la clé SSH — auquel cas il **coupe** l'authentification par mot de passe ;
2. écrire la configuration supervisord correspondant au mode ;
3. se remplacer par supervisord.

Il sert aussi d'outil : la plateforme l'appelle pour changer le mot de passe d'une machine à chaud, sans la redémarrer.

```
docker exec <conteneur> /usr/local/bin/entrypoint.sh setpass <nouveau>
```

Ce que ça a changé

Une seule recette pour six variantes, et la rotation du mot de passe sans interruption de service.

Ce que la plateforme sait faire après la couche 3

Elle fabrique de vraies machines Linux, accessibles de trois façons, durcies, et qui se réparent toutes seules en interne. Mais quelqu'un doit encore décider *quand* les créer et *quand* les détruire — et sur quel serveur.

CHAPITRE 5

Couche 4 — Le pilotage du parc

Les machines existent, mais rien ne décide quand les créer, où les placer, ni quand les détruire. Sept briques pour transformer un démon Docker en un service de location.

subprocess

brique 22 · piloter Docker depuis Python

C'est quoi

Le module de la bibliothèque standard qui lance un programme externe et récupère sa sortie. Ici, il lance la commande `docker`.

Pourquoi celui-là

Il existe une bibliothèque Python officielle, `docker-py`, qui parle à l'API Docker sans passer par la ligne de commande. Elle est plus élégante.

On l'a écartée pour une raison de **conformité et de lisibilité**. Le sujet impose une commande précise :

```
docker run -d --restart=always -p <port>:22 <image>
```

Avec un SDK, cette commande n'apparaît nulle part : elle devient un appel de méthode avec des paramètres nommés. Avec `subprocess`, chaque option se lit telle quelle dans le code, et l'on peut montrer à la prof la ligne exacte qui part.

Son rôle ici

Une seule fonction centralise tous les appels, dans `2_webapp/docker_ops.py` :

```
def run_docker(*args, worker="local", timeout=DOCKER_TIMEOUT) -> str:
    cmd = [*wk.docker_command(worker), *args]
    # Appel sans shell : les arguments sont passes en liste, donc
        ↪ jamais
    # re-interpretes par un interpreteur de commandes.
    res = subprocess.run(cmd, capture_output=True, text=True, timeout=
        ↪ timeout)
```

Piège — `shell=True` est une porte ouverte

Écrire `subprocess.run(f"docker run ... {nom}", shell=True)` laisse-

rait un utilisateur nommer sa machine `x; rm -rf /` et faire exécuter n'importe quoi au serveur. C'est une *injection de commande*.
Passer une **liste** d'arguments, sans shell, supprime le problème à la racine : il n'y a aucun interpréteur pour relire la chaîne. Chaque élément de la liste est un argument, point.

Ce que ça a changé

La commande imposée est visible et vérifiable. Et les tests peuvent espionner cette unique fonction pour vérifier ce qui part, sans Docker.

Les étiquettes Docker

brique 23 · l'identité des machines

C'est quoi

Des paires clé-valeur attachées à un conteneur à sa création, et sur lesquelles on peut filtrer.

Pourquoi celui-là

C'est la brique née d'un incident. Le Faucheur doit savoir quels conteneurs lui appartiennent. La première version les reconnaissait à leur nom, qui commence par `insacloud_`.

Puis la pile `docker compose` est arrivée, avec des conteneurs nommés `insacloud-web`, `insacloud-db`, `insacloud-faucheur`. Le Faucheur les a pris pour des machines louées orphelines et les a détruits. **Il s'est supprimé lui-même.**

Son rôle ici

Deux étiquettes, apposées à toute machine louée :

```
MACHINE_LABEL = "insacloud.role"          # = "rented-machine"
CLUSTER_LABEL = "insacloud.cluster"       # = nom du deploiement
```

Les deux filtres sont combinés : une machine doit être une machine louée **et** appartenir à ce déploiement. Le second a été ajouté après un deuxième incident : deux déploiements partageant un démon se détruisaient mutuellement leurs conteneurs.

À retenir

Un nom est une commodité d'affichage ; une étiquette est une identité. Dès qu'un programme doit décider de **détruire** quelque chose, il doit se fonder sur une marque qu'il a lui-même apposée, jamais sur une convention de nommage.

Ce que ça a changé

Le Faucheur ne peut plus se tromper de cible. Deux plateformes peuvent cohabiter sur un même démon Docker.

Le transport SSH de Docker

brique 24 · piloter les nœuds distants

C'est quoi

Le client Docker sait parler à un démon distant à travers une connexion SSH : `docker -H ssh://utilisateur@machine ps`.

Pourquoi celui-là

L'autre façon de commander un démon distant est d'exposer son API sur le réseau, port 2375 en clair ou 2376 en TLS. C'est plus direct, et c'est une très mauvaise idée : **qui parle au démon Docker d'une machine en est root**. Une erreur de pare-feu, et toute la plateforme est compromise.

Le transport SSH ne demande aucun port supplémentaire, réutilise une authentification par clé déjà éprouvée, et chiffre tout.

Son rôle ici

Une fonction construit le préfixe de commande selon le nœud visé :

```
def docker_command(worker: str = "local") -> list:
    host = worker_host(worker)
    if host is None:
        return ["docker"]
    return ["docker", "-H", f"ssh://{DOCKER_SSH_USER}@{host}"]
```

Conséquence heureuse : **le code est le même** en mono-hôte et en cluster. Seul le préfixe change. C'est ce qui permet de développer sur un portable sans monter quatre machines virtuelles.

Ce que ça a changé

Un parc distribué sans ouvrir un seul port Docker sur le réseau.

L'agent des nœuds

brique 25 · l'inscription spontanée

C'est quoi

Un programme Python de 263 lignes qui tourne sur chaque worker. Au démarrage il se présente au contrôleur ; ensuite il signale sa présence toutes les trente secondes.

Pourquoi celui-là

Sans lui, il faudrait écrire la liste des workers dans la configuration du contrôleur. Ajouter un nœud deviendrait : modifier un fichier, redéployer.

Et un nœud en panne resterait dans la liste jusqu'à ce qu'un humain l'en retire.

L'agent inverse la responsabilité : c'est le nœud qui se déclare. Le contrôleur ne contient **aucune adresse de worker**.

Il n'utilise que la bibliothèque standard — `urllib` pour les appels, `/proc/meminfo` pour la mémoire — précisément pour qu'il n'y ait rien à installer sur les workers.

Son rôle ici

Deux routes de l'API : `POST /workers/register` et `POST /workers/heartbeat`, protégées par un jeton partagé comparé en temps constant. L'inscription est retentée avec un recul exponentiel : au démarrage d'un cluster, un worker peut être prêt avant le contrôleur.

État	Signification
`AVAILABLE`	le nœud bat, Docker répond, il reste de la place
`BUSY`	le nœud bat, mais il est plein : on ne lui envoie plus rien
`OFFLINE`	plus de battement depuis 90 s : ses machines sont recréées ailleurs

> **À retenir**
>
> La distinction `BUSY` / `OFFLINE` est essentielle. Un nœud saturé est en bonne santé : migrer ses machines serait coûteux et inutile. Confondre « plein » et « mort » déclencherait des reprises à répétition.

Ce que ça a changé

Pour ajouter un worker, on l'allume. C'est tout. C'est la Fonctionnalité 7 du sujet, et c'est ce qui rend le parc élastique.

L'ordonnanceur

brique 26 · choisir le nœud

C'est quoi

La fonction qui décide sur quel worker placer une nouvelle machine.

Pourquoi celui-là

Les stratégies possibles sont nombreuses : tourniquet, aléatoire, par pondération de ressources. On a choisi la plus simple qui soit défendable — le nœud joignable le **moins chargé** — pour une raison précise : elle est **déterministe**, donc testable.

> **À retenir**
>
> Un ordonnanceur dont le départage en cas d'égalité est aléatoire donne un test intermittent. Et un test intermittent finit toujours par être

ignoré. Ici, l'égalité se tranche par ordre alphabétique.

Son rôle ici

Trois règles : ne considérer que les nœuds `AVAILABLE` ; parmi eux, prendre celui qui héberge le moins de machines ; départager par le nom. Puis réserver un port libre, dont l'unicité est garantie par l'index partiel de la couche 2.

Ce que ça a changé

La charge se répartit, et le comportement est reproductible — donc vérifiable par un test.

Le Faucheur

brique 27 · le démon de nettoyage

C'est quoi

Un processus séparé qui tourne en boucle, toutes les dix secondes, avec trois missions : détruire ce qui a expiré, réconcilier Docker et la base, et surveiller le parc.

Pourquoi celui-là

Personne ne viendra cliquer sur « supprimer » à trois heures du matin. Il faut un service qui surveille les horloges.

Pourquoi un **processus séparé** plutôt qu'un fil d'exécution dans l'application ? Parce que Gunicorn lance quatre processus : il y aurait quatre Faucheurs qui se marcheraient dessus. Un service distinct, démarré une fois par systemd, évite le problème et peut être redémarré sans couper le site.

Son rôle ici

Trois tâches à chaque cycle.

Expirer : une seule requête SQL, celle que sert l'index `(status, end_time)`.

Réconcilier : la base dit ce qui *devrait* exister, Docker ce qui existe *vraiment*. Un conteneur orphelin est supprimé ; une instance sans conteneur passe à « arrêtée ».

Surveiller le parc : déclarer hors ligne les nœuds muets, et faire recréer leurs machines ailleurs.

Piège — Un nœud injoignable n'est pas un nœud vide

C'est le piège le plus dangereux de la réconciliation. Si le nœud ne répond pas, `docker ps` renvoie une liste vide. Un code naïf en conclurait que toutes ses machines ont disparu et les marquerait « arrêtées » — alors qu'elles tournent parfaitement.

D'où la règle : un nœud injoignable est **ignoré**, jamais interprété.

Ce que ça a changé

Les machines disparaissent toutes seules à l'échéance, les ports se libèrent, et la base ne dérive plus de la réalité. C'est la Fonctionnalité 12, et la moitié de la 13.

systemd

brique 28 · faire tourner les services, et les réparer

C'est quoi

Le gestionnaire de services de Linux : il démarre les programmes au bon moment, les redémarre s'ils meurent, et collecte leurs journaux.

Pourquoi celui-là

L'alternative serait de lancer les processus à la main dans un `screen`, ou d'écrire des scripts d'init. systemd apporte trois choses qu'on ne veut pas réécrire : le redémarrage automatique, l'ordre de démarrage selon les dépendances, et les journaux consultables par `journalctl`.

Son rôle ici

Deux unités et un *timer*.

L'unité de l'application lance Gunicorn sous un compte sans privilège, avec les secrets lus dans un fichier séparé :

```
[Service]
User=insacloud
EnvironmentFile=/etc/insacloud/insacloud.env     # secrets, en 0600
ExecStart=/opt/insacloud/.venv/bin/gunicorn --preload -w 4 ...
Restart=always
```

> **Piège — Un secret dans une unité systemd est public**
>
> Mettre `Environment=INSACLOUD_VAULT_KEY=...` dans le fichier d'unité le rend lisible par tous : les unités sont en lecture pour tout le monde, et `systemctl show` les affiche. La bonne façon est un fichier séparé en `0600`, référencé par `EnvironmentFile=`.

Le watchdog est un *timer* qui se déclenche chaque minute. Il vérifie que nginx répond, que le site renvoie bien du HTTPS, et que le Faucheur tourne. Il redémarre ce qui est tombé et journalise les workers injoignables.

Pourquoi un timer systemd plutôt que cron ? Parce qu'il s'intègre aux journaux, connaît l'état du service qu'il surveille, et ne démarre pas avant que ses dépendances soient prêtes. Cron ne sait rien de tout cela.

Ce que ça a changé

Le service se relance seul après une panne ou un redémarrage du serveur. C'est le troisième ajout au-delà du sujet.

Ce que la plateforme sait faire après la couche 4

Elle loue, place, surveille et détruit. Elle survit à la panne d'un nœud. Elle se répare elle-même. Mais tout cela n'existe que sur des machines que quelqu'un a installées à la main — et ça, ce n'est pas tenable.

CHAPITRE 6

Couche 5 — L'infrastructure

Jusqu'ici, tout suppose des serveurs déjà installés. Cinq briques pour que ces serveurs se créent et se configurent tout seuls, à l'identique, autant de fois qu'on veut.



Figure 6.1 – Deux commandes séparent un dossier vide d'une plateforme en fonctionnement. Tout ce qui se passe entre les deux est décrit dans des fichiers versionnés.

Vagrant

brique 29 · créer les machines virtuelles

C'est quoi

Un outil qui décrit des machines virtuelles dans un fichier texte et les crée en une commande. `vagrant up` les monte, `vagrant destroy` les efface.

Pourquoi celui-là

L'alternative est de créer les VM à la main dans l'interface de VirtualBox ou de VMware : choisir l'image, la mémoire, le réseau, installer le système. Une demi- heure par machine, et personne ne saura ensuite ce qui a été coché.

Terraform aurait pu faire le travail, mais il vise les fournisseurs de cloud publics ; pour des VM locales, Vagrant est plus direct et c'est l'outil que le sujet nomme.

Son rôle ici

Le `Vagrantfile` décrit un contrôleur en 192.168.56.10 et trois workers en .11, .12, .13, avec leur mémoire et leurs cœurs. Le nombre de workers et les tailles sont réglables sans toucher au fichier :

```
WORKERS=2 CTRL_MEM=1536 WORKER_MEM=1280 vagrant up
```

À retenir

Rendre les tailles paramétrables coûte trois lignes et change tout pour qui reprend le projet : on peut le faire tourner sur un portable modeste. Un fichier qui n'offre qu'une seule taille finit toujours par être modifié

à la main, donc désynchronisé du dépôt.

Ce que ça a changé

Le parc entier se crée en une commande, et se détruit aussi vite. On peut tout casser pour vérifier que tout se remonte.

Les providers : VMware, VirtualBox, Parallels, QEMU

brique 30 · exécuter les VM

C'est quoi

Le *provider* est le logiciel de virtualisation que Vagrant pilote. Vagrant ne virtualise rien lui-même : il donne des ordres à l'un d'eux.

Pourquoi celui-là

Un seul provider aurait suffi... sur une seule architecture. Le projet devait tourner sur des postes Intel et sur des Mac à puce Apple.

VirtualBox est le provider par défaut et le plus répandu, mais son support ARM reste expérimental : sur un Mac récent, il ne convient pas. **VMware Fusion** fonctionne nativement sur Apple Silicon. **Parallels** aussi, mais il est payant. **QEMU** est libre et fonctionne partout, au prix de limitations.

Déclarer les quatre laisse le même fichier fonctionner sur n'importe quel poste.

Son rôle ici

Chaque provider a son bloc dans le `Vagrantfile`, avec ses réglages propres.

Piège — Chaque provider a ses limites

`vagrant-qemu` ne sait pas créer de réseau privé : l'accès SSH doit être redirigé sur `localhost`. Le fichier le détecte et ajuste la configuration. Sans cela, le déploiement réussirait et rien ne serait joignable — la pire sorte de panne, celle qui ne dit pas son nom.

Ce que ça a changé

Le projet se monte sur le poste de n'importe quel membre du groupe, et sur l'ordinateur de la salle de soutenance.

Ansible

brique 31 · configurer les machines

C'est quoi

Un outil qui se connecte en SSH aux machines et applique un **état décrit en YAML** : tel paquet installé, tel fichier présent avec tels droits, tel service démarré.

Pourquoi celui-là

L'alternative est un script shell. La différence n'est pas cosmétique.

Un script décrit des **actions** : installe, crée, copie, redémarre. Le relancer refait tout, écrase, redémarre inutilement, et échoue si quelque chose existe déjà.

Ansible décrit un **état voulu**. Il regarde, compare, et n'agit que si nécessaire. C'est la propriété d'**idempotence** : appliquer deux fois donne le même résultat qu'appliquer une fois.

Les concurrents — Puppet, Chef, Salt — exigent un agent installé sur chaque machine. Ansible n'a besoin que de SSH et de Python, déjà présents. C'est aussi l'outil que le sujet nomme.

> **Notion — Idempotent ne veut pas dire « ne fait rien »**
>
> Cela veut dire « vérifie l'état, et n'agit que s'il diffère de celui qui est voulu ». La nuance compte : c'est elle qui permet de relancer le déploiement sans crainte, à n'importe quel moment, y compris sur un parc à moitié configuré.
>
> La preuve se lit dans le compte rendu : au second passage, `changed=0` sur tous les nœuds.

Son rôle ici

Neuf rôles, quatre playbooks. Chaque rôle est autonome, avec ses variables par défaut, ses fichiers et ses gabarits.

Rôle	Ce qu'il garantit
`security_hardening`	le pare-feu UFW, avec les ports du groupe
`controller_key`	le compte de service et sa clé SSH de pilotage
`docker`	Docker Engine sur les workers, le client seul sur le contrôleur
`tls_cert`	un certificat propre à chaque nœud
`worker`	l'accès SSH restreint du contrôleur
`docker_images`	les images des machines louées, construites sur place
`worker_agent`	l'agent d'inscription, en service systemd
`reverse_proxy`	nginx en HTTPS devant Gunicorn
`webapp`	l'application, le Faucheur, le watchdog, les secrets

> **Piège — Un rôle appelé avec des paramètres différents s'exécute deux fois**
>
> Ansible dédoublonne les rôles... sauf si on les appelle avec des paramètres différents : il les considère alors comme distincts. Le rôle `docker`

était appelé directement *et* tiré en dépendance par `docker_images`, avec un paramètre différent — il s'exécutait deux fois, installant puis désinstallant le démon.
La correction : passer par une **variable de play**, qui s'applique aux deux appels et les fait fusionner. Nous l'avons vérifié par l'expérience, en instrumentant le rôle pour qu'il journalise chaque exécution.

Ce que ça a changé

Quatre machines nues deviennent une plateforme en fonctionnement, en une commande, à l'identique. Et le déploiement peut être rejoué sans risque.

Ansible Vault

brique 32 · les secrets dans Git

C'est quoi

Un fichier chiffré en AES-256 qu'Ansible déchiffre au moment de jouer le playbook, avec une phrase de passe fournie à l'exécution.

Pourquoi celui-là

Les secrets — clé de session, clé du coffre, mot de passe de la base, jeton des agents — doivent arriver sur le serveur. Trois mauvaises façons de s'y prendre :

Les écrire en clair dans le dépôt : `gitleaks` les trouverait, et n'importe qui ayant accès au dépôt aussi.

Les passer en variables d'environnement à la main : il faut s'en souvenir, et ils finissent dans l'historique du shell.

Les mettre dans un gestionnaire externe — HashiCorp Vault, AWS Secrets Manager : la bonne solution en entreprise, mais c'est un service de plus à déployer et à maintenir.

Ansible Vault est le compromis juste pour ce projet : un fichier, **versionnable tel quel**, illisible sans la phrase de passe.

Son rôle ici

`3_ansible/group_vars/insacloud/vault.yml` contient trois secrets. Le rôle `webapp` vérifie même leur forme avant de déployer — la clé Fernet doit faire exactement 44 caractères — plutôt que de laisser l'application échouer au démarrage.

La phrase de passe vit dans un fichier `.vault_pass` en `0600`, exclu du dépôt par le `.gitignore`.

Ce que ça a changé

Les secrets voyagent avec le code sans jamais être lisibles. Et le déploiement reste automatisable : pas d'étape manuelle de copie de clés.

L'inventaire dynamique brique 33 · découvrir les machines

C'est quoi

Un script exécutable qui, au lieu de lire une liste figée de machines, interroge Vagrant et produit l'inventaire en JSON.

Pourquoi celui-là

L'inventaire statique est un fichier qui liste les machines et leurs adresses. Il a deux défauts : il faut le tenir à jour quand le parc change, et il code en dur le chemin de la clé SSH — qui dépend du provider.

C'est un piège réel : l'inventaire du projet contenait `vagrant_provider=virtualbox`. En passant à VMware, Ansible cherchait la clé au mauvais endroit.

Son rôle ici

`3_ansible/inventaire_vagrant.py` interroge `vagrant ssh-config` et reprend le chemin de clé que Vagrant *déclare lui-même*. Il ne découvre que les machines réellement démarrées.

```
ansible-playbook -i inventaire_vagrant.py site.yml --ask-vault-pass
```

Ce que ça a changé

Le même dépôt se déploie sur les quatre providers sans modification, et sur un parc de taille variable. C'est le quatrième ajout au-delà du sujet.

Ce que la plateforme sait faire après la couche 5

Elle se construit entièrement toute seule : quatre machines virtuelles, leur système, leurs paquets, leurs certificats, leurs services, ses secrets. Deux commandes depuis un dossier vide.

Il reste à garantir qu'elle ne se dégrade pas.

CHAPITRE 7

Couche 6 — La qualité

La plateforme marche. Reste à garantir qu'elle marchera encore après la prochaine modification — et qu'elle ne livre pas de faille connue. Treize briques, dont onze tournent à chaque envoi de code.

pytest

brique 34 · les tests

C'est quoi

Le cadre de test de Python. Une fonction dont le nom commence par `test_` est un test ; une assertion fausse le fait échouer.

Pourquoi celui-là

L'alternative dans la bibliothèque standard est `unittest`, qui impose d'écrire des classes et des méthodes à noms imposés. pytest accepte des fonctions simples, et surtout il apporte les *fixtures* : des morceaux de préparation réutilisables, dont l'un est central ici.

Son rôle ici

168 tests, en vingt secondes, répartis en six fichiers. Deux mécanismes font tout le travail.

La base vierge à chaque test. Une fixture marquée `autouse=True` efface et recrée la base avant chaque test, sans qu'on ait à y penser.

> **Piège — Des tests qui dépendent de leur ordre**
>
> Si un test laisse des données derrière lui, le suivant peut réussir pour la mauvaise raison — ou échouer quand on le lance seul. Le symptôme : « ça passe en entier mais pas tout seul », ou l'inverse. C'est le genre de bug qui fait abandonner une suite de tests.

L'espion sur Docker. Les tests ne lancent aucun conteneur : `run_docker` est remplacé par une fonction qui note ce qu'on lui demande. On vérifie ainsi que la commande imposée par le sujet part bien telle quelle, sur une machine qui n'a même pas Docker.

Ce que ça a changé

Les tests tournent partout, en vingt secondes. C'est ce qui permet de les lancer à chaque envoi plutôt qu'une fois par mois.

ruff

brique 35 · l'analyse de style et d'erreurs

C'est quoi

Un analyseur qui lit le code sans l'exécuter et signale ce qui est douteux : import jamais utilisé, variable inconnue, tournure obsolète, motif dangereux.

Pourquoi celui-là

Les outils historiques sont `flake8` pour le style et `pylint` pour l'analyse, plus `isort` pour les imports. Ruff fait les trois, est écrit en Rust, et tourne en moins d'une seconde sur tout le projet — contre plusieurs dizaines pour pylint. Cette vitesse change l'usage : on le lance avant chaque commit au lieu de l'éviter.

Son rôle ici

Configuré dans `pyproject.toml`, avec six familles de règles sélectionnées. Une seule est désactivée — la longueur de ligne — parce que les commentaires pédagogiques du projet dépassent parfois.

Il vérifie aussi `target-version = "py310"` : il refuse les tournures que le Python de la machine cible ne comprendrait pas.

Ce que ça a changé

Le code reste lisible et cohérent sans discussion de style, et les fautes d'inattention sont attrapées avant l'exécution.

bandit

brique 36 · l'analyse de sécurité du code (SAST)

C'est quoi

Static Application Security Testing : il cherche dans le code source des motifs connus pour être dangereux.

Pourquoi celui-là

Un relecteur humain oublie. bandit connaît une liste de pièges classiques et ne se fatigue pas : concaténation SQL, `shell=True`, secret en dur, algorithme de hachage obsolète, fichier temporaire prévisible.

Son rôle ici

Lancé avec `--severity-level low --confidence-level low` : on veut tout voir. Le projet ne désactive **aucune règle globalement** ; les rares cas légitimes sont annotés ligne à ligne avec leur justification — par exemple `# nosec B404` sur l'import de `subprocess`, puisque le sujet impose de piloter Docker par sa ligne de commande.

Piège — Le témoin de vie dans `/tmp`

Le Faucheur horodate un fichier à chaque cycle, pour que la sonde du conteneur sache que sa boucle tourne. Nous l'avions mis dans `/tmp` ; bandit a signalé la règle B108, et il avait raison.
Le Faucheur tourne aussi en service systemd sur le contrôleur, où `/tmp` est partagé avec tous les comptes locaux : un lien symbolique y suffirait à nous faire horodater un fichier qui ne nous appartient pas. Le témoin vit désormais dans le dossier de l'application.

Ce que ça a changé

Une famille entière d'erreurs est attrapée automatiquement, y compris celles qu'on vient d'introduire sans y penser.

pip-audit

brique 37 · l'audit des dépendances (SCA)

C'est quoi

Software Composition Analysis : il confronte la liste des dépendances épinglées aux bases de vulnérabilités publiques.

Pourquoi celui-là

On écrit soi-même quelques milliers de lignes, mais on en importe des centaines de milliers. Une faille dans Flask ou dans Werkzeug est une faille du projet, et on ne l'apprend pas en relisant son propre code.

L'alternative est `safety`, dont la base complète est payante. pip-audit est publié par l'autorité de packaging Python et utilise les bases publiques.

Son rôle ici

Lancé sur `requirements.txt` avec `--strict`. C'est la brique qui rend l'épinglage des versions (brique 3) exploitable : on ne peut auditer que ce qu'on connaît exactement.

À retenir

Ce travail trouve des failles **sans qu'on ait rien changé**. Werkzeug 3.1.8 est devenu vulnérable (CVE-2026-102598) entre deux exécutions de la chaîne : notre code n'avait pas bougé, le monde si. Sans ce contrôle automatique, nous l'aurions appris trop tard.

Ce que ça a changé

Les failles des dépendances sont vues le jour de leur publication, pas le jour de l'incident.

gitleaks

brique 38 · la recherche de secrets

C'est quoi

Un outil qui parcourt le dépôt **et tout son historique** à la recherche de clés, jetons et mots de passe.

Pourquoi celui-là

C'est la brique qui corrige une illusion : effacer un secret dans un commit ultérieur ne l'efface pas du dépôt. Il reste dans l'historique, consultable par quiconque clone.

Son rôle ici

Lancé sur l'historique complet à chaque envoi. C'est ce qui permet au Vault d'être versionné sans crainte : s'il était un jour commis en clair, la chaîne le verrait immédiatement.

Ce que ça a changé

Une erreur humaine — commettre un fichier de configuration contenant un mot de passe — est détectée avant qu'elle ne devienne publique.

nuclei et dast.sh

brique 39 · l'analyse dynamique (DAST)

C'est quoi

Dynamic Application Security Testing : on attaque l'application **en fonctionnement**, par le réseau, comme le ferait un attaquant.

Pourquoi celui-là

bandit lit le code ; il ne peut pas savoir si l'en-tête de sécurité est réellement envoyé, ni si une page protégée l'est vraiment une fois déployée. Seul un test contre le service qui tourne le dit.

`nuclei` est un scanner générique avec des milliers de signatures. `tests/dast.sh` est un script maison qui vérifie ce que nuclei ne connaît pas : les règles propres à ce projet.

Son rôle ici

La chaîne démarre l'application avec Gunicorn, puis lance les deux. Dix-huit contrôles : en-têtes de sécurité, pages protégées inaccessibles sans session, jetons CSRF exigés, attributs des cookies.

Ce que ça a changé

On vérifie le comportement réel, pas l'intention du code.

hadolint et ansible-lint

brique 40 · l'analyse de l'infrastructure

C'est quoi

Deux analyseurs spécialisés : `hadolint` pour les Dockerfiles, `ansible-lint` pour les rôles Ansible.

Pourquoi celui-là

Le code d'infrastructure est du code, avec ses propres pièges. Un Dockerfile peut figer un mot de passe dans une couche, installer une version non épinglée, ou tourner en root sans raison. Un rôle Ansible peut être non idempotent sans que cela se voie.

Son rôle ici

`hadolint` analyse les quatre Dockerfiles. C'est lui qui a trouvé le mot de passe par défaut de la brique 16.

`ansible-lint` tourne au profil `production`, le plus strict. Il vérifie notamment que les tâches ont un nom, que les modules sont qualifiés (`ansible.builtin.*`), et signale les usages non idempotents.

> **Piège — `command` et `shell` ne sont jamais idempotents**
>
> Ces deux modules exécutent ce qu'on leur donne sans rien vérifier : ils sont donc *toujours* signalés `changed`. Il faut leur apprendre à se taire avec `creates:`, `removes:` ou `changed_when:`.

Ce que ça a changé

L'infrastructure est relue avec la même exigence que le code applicatif.

Trivy

brique 41 · le scan des images

C'est quoi

Un scanner qui inventorie les paquets d'une image Docker et cherche leurs vulnérabilités connues.

Pourquoi celui-là

pip-audit couvre les dépendances Python. Mais une image contient aussi un système de base — Debian, Ubuntu, Alpine — avec ses centaines de paquets. Une faille OpenSSL dans l'image de base est une faille livrée.

Son rôle ici

La chaîne inventorie *toutes* les failles sans bloquer, puis relance Trivy avec un verrou : aucune faille `HIGH` ou `CRITICAL` **corrigeable** ne passe.

> **À retenir**
>
> Le mot « corrigeable » fait tout. L'option `--ignore-unfixed` écarte les failles sans correctif disponible : les refuser ne protégerait de rien et

bloquerait tout, ce qui finirait par faire désactiver le contrôle.

Ce verrou a bloqué nos propres images : la base `docker:28-cli` traînait 163 failles corrigeables liées à sa version de Go. Il a fallu passer à `docker:29-cli` — deux failles au lieu de 163 — et ajouter un `apt-get upgrade` pour OpenSSL.

Ce que ça a changé

Aucune image vulnérable ne part au registre. Et la preuve que le contrôle sert : il nous a refusé les nôtres.

Syft

brique 42 · l'inventaire de ce qu'on livre (SBOM)

C'est quoi

Software Bill of Materials : la liste exhaustive des composants d'une image livrée, avec leurs versions.

Pourquoi celui-là

Quand une faille majeure est annoncée, la première question est : « sommes-nous touchés ? ». Sans inventaire, il faut reconstruire l'image et chercher. Avec un SBOM archivé, on lit un fichier.

C'est aussi une exigence réglementaire croissante pour tout logiciel livré.

Son rôle ici

Généré à chaque publication et conservé en artefact de la chaîne — environ 340 Ko décrivant tout ce que contiennent les images.

Ce que ça a changé

Le projet peut répondre, des mois plus tard, à une question sur une faille publiée après sa livraison.

GitHub Actions

brique 43 · l'orchestration de tout cela

C'est quoi

Le service qui exécute automatiquement une suite de travaux à chaque envoi de code, sur des machines neuves.

Pourquoi celui-là

Les onze contrôles précédents ne valent que s'ils tournent. Lancés à la main, ils sont oubliés le jour où l'on est pressé — c'est-à-dire le jour où l'on en a le plus besoin.

Les alternatives — GitLab CI, Jenkins — auraient demandé un serveur à administrer. GitHub Actions est intégré au dépôt, et son exécution sur

une machine neuve à chaque fois garantit qu'on ne dépend pas d'un reste de configuration locale.

Son rôle ici

Onze travaux dans `.github/workflows/ci.yml`, 601 lignes. Et surtout une **barrière** : la publication et le déploiement ne s'exécutent que si les neuf précédents sont au vert, et seulement sur la branche principale.

> **À retenir**
>
> Sans dépendance déclarée (`needs:`), GitHub Actions lance tous les travaux en parallèle — et publierait l'image pendant que les tests échouent. C'est la dépendance explicite qui transforme une liste de contrôles en véritable barrière.

> **Piège — La chaîne vérifie sa propre syntaxe**
>
> Une erreur de syntaxe shell dans un bloc `run:` ne se voit qu'au moment où l'étape s'exécute, parfois après dix minutes. Trois exécutions ont échoué pour deux guillemets déséquilibrées.
> Un contrôle a été ajouté : il extrait chaque bloc `run:`, neutralise les expressions de modèle, et passe le tout à `bash -n` avant que quoi que ce soit ne démarre.

Ce que ça a changé

Onze contrôles qui tournent vraiment, à chaque fois, sur une machine neuve.

ghcr.io et docker compose

brique 44 · livrer et faire tourner

C'est quoi

ghcr.io est le registre d'images de GitHub : l'endroit où les images construites sont publiées. **docker compose** décrit une pile de conteneurs dans un fichier et la démarre en une commande.

Pourquoi celui-là

Pour le registre : il est intégré au dépôt, l'authentification est celle de GitHub Actions, et les images sont versionnées avec le commit qui les a produites.

Pour compose : monter quatre machines virtuelles demande du temps et de la mémoire. Pour développer, pour démontrer, et pour faire tourner les tests d'intégration dans la chaîne, on veut la pile complète en une commande.

Son rôle ici

Les images sont étiquetées trois fois : par version, par commit, et `latest`.

Le fichier `docker-compose.yml` décrit la base, l'application, le Faucheur, et les six constructions d'images sous un profil séparé.

Deux mécanismes y méritent d'être signalés.

L'ancre YAML : un bloc nommé une fois, repris par l'application et le Faucheur. Il devient impossible qu'ils divergent sur une variable.

Le refus de démarrer sans secret : la syntaxe `${POSTGRES_PASSWORD:?manquant}` fait échouer le démarrage si la variable n'est pas définie. L'erreur arrive tout de suite, pas trois semaines plus tard.

> **Piège — Un volume par service là où il en fallait un partagé**
>
> Le Faucheur *écrit* les archives, l'application les *sert* au téléchargement. Avec un volume par service, l'utilisateur verrait dans son tableau de bord une archive que le serveur ne trouverait pas. Le même volume nommé doit être monté dans les deux.



Figure 7.1 – Les archives servies par l'application, écrites par le Faucheur : deux services, un seul volume.

Ce que ça a changé

Le projet se livre et se lance en une commande, en développement comme en intégration continue.

Ce que la plateforme sait faire après la couche 6

Elle se vérifie elle-même. Onze contrôles à chaque envoi, dont six de sécurité, et aucune livraison si l'un échoue. Elle a trouvé dix défauts réels que la relecture n'avait pas vus.

CHAPITRE 8

Récapitulatif

8.1 Les quarante-quatre briques, d'un coup d'œil

#	Outil	Son rôle ici	Écarté au profit de lui
Couche 0 — Le socle			
1	Git	historique, branches, base de la CI	copier le dossier
2	Python 3.10	tout le code, version de la cible	Node, Go (agent à déployer)
3	venv + épinglage	versions identiques partout	installation globale
Couche 1 — L'application web			
4	Flask	routes HTML et JSON, sessions	Django (trop de boîtes noires)
5	Jinja2	gabarits, échappement automatique	concaténation de chaînes
6	Werkzeug	hachage pbkdf2, ProxyFix	hachage écrit à la main
7	Gunicorn	4 ouvriers, production	serveur de dév. de Flask
8	nginx	TLS, statiques, en-têtes	Apache, Caddy
9	Tailwind précompilé	apparence sans CDN	Tailwind par CDN (interdit par la CSP)
Couche 2 — Les données			
10	SQLAlchemy	7 tables, pas d'injection SQL	SQL à la main
11	Alembic	migrations versionnées	`ALTER TABLE` manuels
12	SQLite	développement et tests	un serveur à lancer
13	PostgreSQL	production, index partiels	MySQL
14	Fernet	coffre des mots de passe root	chiffrement maison
Couche 3 — Les machines louées			
15	Docker	machines en moins d'une seconde	machines virtuelles
16	Dockerfile	6 images reproductibles	image figée à la main
17	supervisord	5 services, PID 1 correct	script shell, systemd
18	OpenSSH	l'accès imposé par le sujet	—
19	ttyd	terminal sans rien installer	Shellinabox, Wetty
20	Xvnc + XFCE + noVNC	bureau dans le navigateur	GNOME, KDE (trop lourds)
21	entrypoint.sh	assemble la machine selon le mode	six images distinctes
Couche 4 — Le pilotage			
22	subprocess	la commande imposée, visible	SDK docker-py
23	étiquettes Docker	identité des machines louées	préfixe de nom
24	`docker -H ssh://`	piloter sans ouvrir de port	API Docker exposée
25	l'agent	inscription et battement de cœur	liste de workers en dur
26	l'ordonnanceur	nœud le moins chargé, déterministe	tirage aléatoire
27	le Faucheur	expiration, réconciliation, reprise	fil dans l'application
28	systemd + watchdog	services relancés, autoréparation	cron, lancement manuel
Couche 5 — L'infrastructure			
29	Vagrant	4 VM en une commande	création à la main
30	VMware et 3 autres	fonctionne sur Intel et Apple	VirtualBox seul
31	Ansible	9 rôles idempotents	scripts shell
32	Ansible Vault	secrets versionnables	secrets en clair, Vault externe
33	inventaire dynamique	découvre les VM et leur clé	inventaire figé
Couche 6 — La qualité			
34	pytest	168 tests en 20 s, sans Docker	unittest
35	ruff	style et erreurs, en moins d'1 s	flake8 + pylint + isort
36	bandit	motifs dangereux (SAST)	relecture humaine
37	pip-audit	CVE des dépendances (SCA)	safety (base payante)
38	gitleaks	secrets, historique compris	recherche manuelle
39	nuclei + dast.sh	l'application en marche (DAST)	analyse statique seule
40	hadolint + ansible-lint	l'infrastructure relue	relecture humaine
41	Trivy	CVE des images, verrou	publication sans scan
42	Syft	SBOM conservé	aucun inventaire

8.2 Qui réalise quoi dans le cahier des charges

#	Fonctionnalité	Briques mobilisées
1	Préparation, environnement	Git · Python · venv
2	API Web Flask	Flask · Jinja2 · Werkzeug · Gunicorn · nginx · Tailwind
3	Base de données	SQLAlchemy · Alembic · SQLite · PostgreSQL
4	Conteneurisation Docker	Docker · Dockerfile · supervisord · entrypoint
5	Infrastructure Vagrant	Vagrant · les quatre providers
6	Automatisation Ansible	Ansible · Vault · inventaire dynamique
7	Worker Agent	l'agent · les étiquettes · l'API des nœuds
8	Resource Manager	l'ordonnanceur · l'index partiel de PostgreSQL
9	Service de location	Flask · services.py · SQLAlchemy
10	Déploiement des instances	subprocess · Docker · `docker -H ssh://`
11	Accès SSH	OpenSSH · ttyd · Xvnc + XFCE + noVNC
12	Gestion de l'expiration	le Faucheur · l'index `(status, end_time)`
13	Haute disponibilité	supervisord · `restart=always` · live-restore · l'agent · le Faucheur
14	Sécurité DevSecOps	Fernet · bandit · pip-audit · gitleaks · nuclei · Trivy · Vault
15	CI/CD	GitHub Actions · ghcr.io · Syft
16	Tests d'intégration	pytest · docker compose · integration.sh
17	Documentation	les schémas engendrés · ce document · le guide de 93 pages

À retenir

Aucune fonctionnalité n'est portée par un seul outil. La haute disponibilité, par exemple, naît de cinq briques posées dans trois couches différentes : supervisord dans la machine, `--restart=always` et `live-restore` sur le nœud, l'agent et le Faucheur dans le parc. C'est pour cela qu'on ne peut pas « ajouter la résilience à la fin ».

8.3 Les sept ajouts au-delà du sujet

Ajout	Briques qui le rendent possible
Parc distribué à trois workers, alors qu'un hôte suffisait	l'agent · l'ordonnanceur · `docker -H ssh://`
Terminal web, en plus du SSH demandé	ttyd · supervisord
Bureau graphique complet dans le navigateur	Xvnc · XFCE · noVNC · supervisord
Coffre à secrets réaffichable après reauthentification	Fernet · les sessions Flask · Ansible Vault
Watchdog d'auto-réparation du service	systemd · un timer d'une minute
Inventaire dynamique indépendant du provider	Vagrant · un script Python de 140 lignes
Sauvegarde du travail avant destruction	Docker `exec` · `tar` en flux · une table de plus

8.4 Ce qu'il faut retenir

Un outil se comprend par ce qu'on a écarté. Savoir qu'on utilise Gunicorn ne vaut rien ; savoir que le serveur de Flask traite une requête à la fois, si. Chaque fiche de ce document a une section « pourquoi celui-là » pour cette raison.

Les couches ne sont pas interchangeables. On ne peut pas conteneuriser avant d'avoir une application, ni automatiser le déploiement avant d'avoir quelque chose à déployer. L'ordre dans lequel les briques ont été posées n'est pas un récit : c'est une contrainte.

Les outils de la couche 6 ne servent à rien s'ils ne bloquent pas. Un contrôle qui affiche un avertissement que personne ne lit est un coût sans bénéfice. Ici, aucune image n'atteint le registre si l'un des onze travaux échoue — et c'est arrivé à nos propres images.

Les meilleures briques viennent d'une panne. Les étiquettes Docker existent parce que le Faucheur s'est supprimé lui-même. Le témoin de vie existe parce que la sonde du conteneur ne pouvait pas passer. L'inventaire dynamique existe parce que l'inventaire figé codait le provider en dur. Un projet sans incident n'a pas de garde-fous.

Dépôt : https://github.com/aziz3r/insacloud
Pour la construction du projet pas à pas, voir le guide technique de 93 pages.